

接下来里是对于代码的复现：

```
1  """
2  Constraint-Aware Hard Action Masking for EVRP-TW
3  Based on: TERRAN (Tang et al., IEEE T-ASE 2026), Section IV-C
4  """
5
6  import numpy as np
7
8
9
10
11  SPEED = 1.0      # 恒定速度 (unit distance / min)
12  ETA = 1.0        # 能耗率: 1 unit distance -> 1 unit energy
13  CAPACITY = 10    # 车辆最大载重
14  BATTERY = 100    # 电池容量 (SoC_max)
15  SERVICE_TIME = 5 # 客户服务时间 (min)
16
17
18  nodes = {
19      0: (0, 0, 0, 0, 200, 0), # Depot
20      1: (2, 0, 5, 20, 80, 1), # Customer C1
21      2: (4, 0, 5, 40, 120, 1), # Customer C2
22      3: (6, 0, 0, 0, 200, 2), # Charging Station
23  }
24
25  num_nodes = len(nodes)
26
27
28  dist = np.zeros((num_nodes, num_nodes))
29  for i in nodes:
30      for j in nodes:
31          dist[i][j] = np.linalg.norm(
32              np.array(nodes[i][:2]) - np.array(nodes[j][:2])
33          )
34
35
36  time_mat = dist / SPEED
37
```

这部分主要是设计状态参数，节点定义，距离矩阵和形式时间矩阵，即进行提前准备。

```
class State:
    def __init__(self):
        self.cur_node = 0 # 当前所在节点
        self.clock = 0    # 当前时间
        self.soc = BATTERY # 当前电量
        self.load = CAPACITY # 当前剩余载重
        self.visited = set() # 已服务客户

state = State()

# =====
# 三个基础掩码 (TERRAN Eq.12-16)
# =====

def time_window_mask(state, next_node):
    """
    TERRAN Eq. (12)
    """
    i = next_node
    arrival = state.clock + time_mat[state.cur_node][i]

    if nodes[i][5] == 1: # customer
        e, l = nodes[i][3], nodes[i][4]
        return 1 if arrival <= l else 0
    else:
        return 1 # depot / CS 无时间窗限制

def capacity_mask(state, next_node):
    """
    TERRAN Eq. (13)
    """
    i = next_node
    if nodes[i][5] == 1: # customer
        demand = nodes[i][2]
        return 1 if state.load >= demand else 0
    else:
        return 1

def energy_mask(state, next_node):
    """
    TERRAN Eq. (16) 简化版[3]单步检查[4]不含FFP[5]
    """
    travel_energy = ETA * time_mat[state.cur_node][next_node]
    return 1 if state.soc >= travel_energy else 0

def feasibility_mask(state):
    """
    综合硬约束掩码
    M_t(i) = M_time · M_cap · M_energy
    """
    mask = np.zeros(num_nodes, dtype=int)

    for i in range(num_nodes):
        # 已服务客户不能再访问
        if i in state.visited and nodes[i][5] == 1:
            mask[i] = 0
            continue

        m_time = time_window_mask(state, i)
        m_cap = capacity_mask(state, i)
        m_energy = energy_mask(state, i)

        mask[i] = m_time * m_cap * m_energy

    return mask
```

这两部分是三个基础掩码，也是核心部分。

```
Step 0
Current Node: 0
Clock: 0.0, SoC: 100.0, Load: 10
Feasibility Mask: [1 1 1 1]
Allowed Nodes: [0, 1, 2, 3]
Selected Action: 0

Step 1
Current Node: 0
Clock: 0.0, SoC: 100.0, Load: 10
Feasibility Mask: [1 1 1 1]
Allowed Nodes: [0, 1, 2, 3]
Selected Action: 0

Step 2
Current Node: 0
Clock: 0.0, SoC: 100.0, Load: 10
Feasibility Mask: [1 1 1 1]
Allowed Nodes: [0, 1, 2, 3]
Selected Action: 0

Step 3
Current Node: 0
Clock: 0.0, SoC: 100.0, Load: 10
Feasibility Mask: [1 1 1 1]
Allowed Nodes: [0, 1, 2, 3]
Selected Action: 0
```

主程序的输出如图所示。

```
PS C:\Users\58083\OneDrive - The University of Nottingham Ningbo China\桌面\科研复现代码> python test_mask.py
✅ Time window mask test passed
✅ Capacity mask test passed
✅ Energy mask test passed
✅ Charging station accessibility test passed
🎉 All tests passed! Masking logic is correct.
```

针对时间窗，容量，能源，充电测试均通过。